Soliman *et al. Journal of Electrical Systems and Information Technology* (2025) 12:95
https://doi.org/10.1186/s43067-025-00288-9

Journal of Electrical Systems and Information Technology

RESEARCH Open Access

# Size matters less: how fine-tuned small LLMs excel in BPMN generation

Ghada Soliman[1,2*], Nour Abdel Wars[1,2] and Hagar Hesham[1,2]

*Correspondence:
Ghada Soliman
ghada.soliman@orange.com;
ghada.soliman@gmail.com
[1]Cairo, Egypt
[2]Department of Artificial Intelligence, Orange Innovation Egypt, Cairo, Egypt

**Abstract**

The generation of Business Process Model and Notation (BPMN) XML outputs from textual process descriptions presents a promising application for large language models (LLMs), yet it introduces significant challenges due to the structured and precise nature of process modeling. This study evaluates the performance of LLMs—Mistral, GPT-4o, Gemini 1.5 Pro, and Claude 3.5 Sonnet—in BPMN generation, employing prompt engineering strategies across Simple, Medium, and Complex process descriptions to establish a baseline. Our findings reveal key limitations in LLMs, including limited output control, input presentation dependencies, and a lack of explainability, particularly for complex processes with nested flows and intricate dependencies. To address these challenges, we propose a novel Description-to-DOT pipeline utilizing a fine-tuned Qwen2.5 14B Coder model, trained on the MaD dataset of process description-DOT representation pairs. The novelty of the Description-to-DOT pipeline lies in its use of Graphviz DOT format as an intermediate representation, which requires generating fewer tokens and enables faster completion, followed by a Python script that converts DOT to BPMN XML in milliseconds—a significant efficiency improvement over the direct Description-to-BPMN pipeline, with the Description-to-DOT pipeline being approximately 6 times faster for Medium processes and 11 times faster for Complex processes. Experimental results demonstrate that the fine-tuned model significantly outperforms the evaluated LLMs, achieving accurate BPMN generation across all complexity levels. This study contributes: (1) Identification of LLM limitations in BPMN generation, such as logical inconsistencies, (2) A novel Description-to-DOT pipeline enhancing efficiency and accuracy, (3) A new benchmark dataset from the MaD dataset for Description-to-BPMN tasks, and (4) Comprehensive validation of the approach across complexity levels. These findings demonstrate the transformative potential of fine-tuned SLMs, with the Qwen2.5 Coder 14B enabling a scalable Description-to-DOT pipeline that excels in BPMN automation across complexity levels, validated on the MaD dataset.

**Keywords** BPMN, Graphviz DOT, LLM, SLM, Prompt engineering

## Introduction

Business Process Management (BPM) aims to optimize organizational workflows, maintain consistent performance, and identify opportunities for improvement. A key component of BPM is the creation of process models, such as BPMN diagrams, which

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

provide a visual representation of workflows. These diagrams are created from a variety of information sources, ranging from well-structured process models and event logs to unstructured textual documents. However, manually creating these diagrams is time-consuming and requires specialized expertise. This research addresses the challenge of automatically generating BPMN diagrams from unstructured and often ambiguous textual descriptions provided by stakeholders.

Early research has explored various approaches to automating the generation of process models from textual descriptions using Natural Language Processing (NLP) techniques. These methods relied on rule-based templates and structured extraction techniques to identify process-related information. [9] proposed a method using Ordered Neurons Long Short-Term Memory (ON-LSTM) networks to automatically discover the hierarchical structure of business processes from textual documents. The ON-LSTM architecture leverages a novel recurrent neural network design that encodes latent semantic hierarchies within textual process descriptions, enabling the extraction of meaningful structural relationships between activities. This approach was tested on Process Description Documents (PDDs) collected from practical Robotic Process Automation (RPA) projects, demonstrating its potential to reduce the time and cost associated with BPM implementation. However, while this approach achieved promising results in uncovering latent tree structures, it demonstrated only 32% accuracy when evaluated using graph edit distance. The method struggled with complex process flows due to limitations in input representation and retrieval algorithms.

Expanding on the use of NLP in process model generation, [23] introduced a method for generating BPMN diagrams from textual software requirements, combining language analysis techniques with informal mapping rules. The authors propose a two-stage approach: first, analyzing textual requirements to extract fact types through NLP techniques such as tokenization, POS tagging, and dependency parsing; second, mapping these fact types onto BPMN elements to generate complete BPMN diagrams. This method addresses limitations in previous studies by handling various sentence structures, including simple, compound, complex, and compound-complex sentences, and incorporating all essential BPMN elements like activities, intermediate events, pools, lanes, datastores, and data objects. The study evaluates the accuracy of the proposed method, achieving 92.83% accuracy in generating BPMN diagrams compared to manual expert modeling. While the results demonstrate significant improvements over prior methods, challenges remain in distinguishing semantically similar entities and interpreting sequential versus parallel conjunctions. These limitations highlight the need for more adaptable and scalable solutions, which has driven recent interest in leveraging LLMs.

With the advent of large language models (LLMs), [27] explored their integration into BPM to enhance tasks throughout the entire lifecycle. The authors identified six core research directions, including identification, discovery, analysis, redesign, implementation, and monitoring. These directions emphasize the transformative potential of LLMs in BPM while also underlining the complexities involved in their deployment. The authors stressed that combining LLM capabilities with existing BPM tools could significantly improve process efficiency and user experience. However, they also highlighted challenges such as managing inflated expectations and addressing limitations in current LLM capabilities, emphasizing the need for extensive guidelines to ensure the successful

application of LLMs. To address this, researchers have explored ways to refine interactions with LLMs through prompt engineering techniques.

Busch et al. [2] investigated the role of prompt engineering in enhancing LLM performance for BPM-related tasks, highlighting that it can mitigate the challenges posed by limited training data, such as process extraction and activity recommendation. The study identifies key benefits, including reduced dependence on extensive labeled datasets, and increased interpretability. However, the authors also acknowledge challenges, such as representing complex process data in prompts, limited prompt length, processing model outputs, and evaluating prompt templates. Despite these advancements, reliance solely on prompt engineering remains insufficient to fully mitigate issues like hallucinations, where LLMs generate inaccurate or irrelevant content when dealing with complex or ambiguous textual descriptions.

Grohs et al. [8] explored the application of LLMs in BPM by mining Imperative and declarative process models with assessing suitability for RPA. The authors utilize LLMs to extract imperative process models from textual descriptions. This involves identifying sequences of activities and their dependencies within business processes. LLMs are applied to derive declarative process models from textual descriptions. This method focuses on capturing constraints and rules that govern the execution of business processes. The study evaluates the potential of LLMs to determine which process tasks described in text are suitable for automation using RPA. The results demonstrate that LLMs can perform BPM tasks effectively, often matching or surpassing existing BPM-specific solutions without extensive configuration or prompt engineering. The versatility of LLMs, in handling multiple BPM tasks, suggests their potential as a general-purpose tool in BPM, indicating promising directions for future research in leveraging LLMs for more complex and varied process-related tasks.

Busch and Leopold. [3] addressed the gap in benchmarking LLMs specifically for BPM tasks. While LLMs are increasingly deployed across various domains, their performance in BPM tasks has not been systematically assessed. The authors compare LLM performance on four BPM tasks, focusing on small open-source models. The analysis aims to identify task-specific performance variations, compare the effectiveness of open-source versus close-source models, and assess the impact of model size on BPM task performance. The findings provide insights into the practical applications of LLMs in BPM, guiding organizations in selecting appropriate models for their specific needs.

Nour Eldin at al. [18] introduced Nala2BPMN, a web-based tool implementing a hybrid approach that decomposes the BPMN model generation task into structured phases to automate the creation of business process models from textual descriptions using LLMs. LLMs analyze and refine textual descriptions, extract process entities (e.g., activities, start/end events) and relationships (e.g., dependencies, branching), while a structured algorithm constructs the final BPMN model, enhancing robustness by reducing hallucinations and logical inconsistencies common in one-step generation methods. The approach begins with prompt engineering to reformulate and complete the textual description, where the LLM refines user-provided inputs by adopting standard terminology, filling in missing details, and ensuring completeness. The reformulated description is then passed to modularized extraction steps for activities, events, dependency relations (forming a directed graph), and branching relations (e.g., parallel or exclusive), with interactivity incorporated at key stages to allow users to validate or update

extracted information. Once the entities are extracted, a structured algorithm translates the dependency graph and branching relations into BPMN elements, including handling loops, cycles, nesting, and skips, to ensure the model aligns closely with the textual description while adhering to BPMN standards. Despite its promising performance in handling simple, complex, and incomplete descriptions, Nala2BPMN relies on resource-intensive closed-source models like GPT-4o, which can limit accessibility and scalability, highlighting the need for more robust and reliable solutions incorporating open-source alternatives or optimizations.

Kourani et al. [13] presented the ProMoAI that leverages LLMs to automate the generation of process models from textual descriptions based on a multi-step approach. In contrast to LLM-based methods that generate process models via prompt engineering in a single step, it directs the LLM to translate natural language inputs into Python code to securely generate Partially Ordered Workflow Language (POWL) models through advanced prompt engineering, including role prompting, knowledge injection, few-shot learning, and negative prompting. These POWL models are subsequently converted into Petri nets and BPMN models, ensuring compatibility with standard process modeling notations. The tool also incorporates user feedback, enabling iterative refinement to optimize and improve the process model. However, it is currently limited to the control-flow perspective and struggles with very complex process descriptions, which can lead to incomplete or inaccurate models.

Licardo et al. [16] present BPMN Assistant, a tool that generates BPMN diagrams from natural language descriptions by leveraging LLMs to produce a structured JSON representation as an intermediate format, which includes elements like tasks, gateways, and events; this JSON is then converted to BPMN XML and enhanced with automatic layout coordinates using the bpmn-auto-layout library for visualization. The generation process involves classifying user inputs as operational intents, prompting the LLM to create the JSON diagram directly from the description, validating it for structural integrity, and rendering it in a web-based interface, with support for iterative editing via natural language instructions that invoke specific functions (e.g., add_element, redirect_branch) to modify the JSON before reconversion to XML. However, the tool is limited to a subset of BPMN elements (e.g., excluding pools and lanes), depends on LLM quality leading to performance variations, lacks semantic evaluation or usability studies, and struggles with ambiguous inputs.

In this study, we investigated the capacity of LLMs, such as Mistral, GPT-4o, Gemini 1.5 Pro, and Claude 3.5 Sonnet, to generate BPM outputs based on textual descriptions. Our analysis revealed that these models struggled to produce complex BPM representations, a limitation potentially stemming from the absence of benchmarks specifically designed to assess LLM performance within the BPM domain. In contrast, fine-tuning the Qwen2.5 14B Coder model enabled the successful generation of sophisticated BPMN outputs. Harnessing its exceptional coding capabilities, this model adeptly handled the intricacies of BPM tasks, underscoring its distinct suitability for this specialized application.

We also employed a systematic workflow to transform textual process descriptions into BPMN representations, leveraging the Business Process Model and Textual Description (MaD) dataset [15] as the primary data source for fine-tuning. The workflow comprises two distinct pipelines—Description-to-DOT and Description-to-BPMN—each

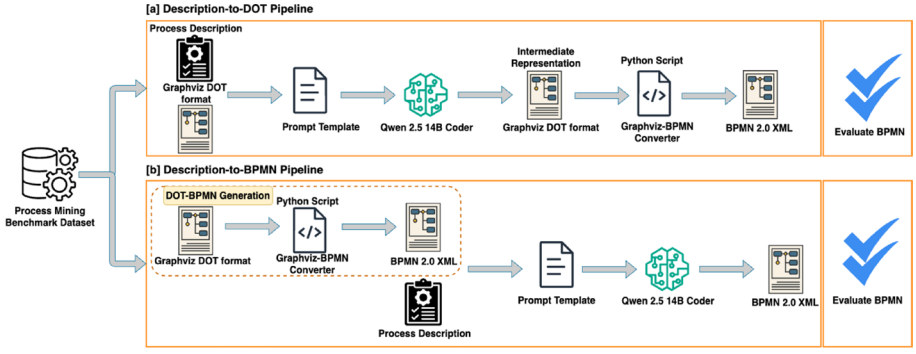



**Fig. 1** Workflow of BPMN generation. (**a**) Generation of graphviz DOT from description (**b**) Generation of BPMN from description

designed to enhance data usability and representation efficiency through tailored approaches as shown in Fig. 1. The Description-to-DOT pipeline utilizes Graphviz DOT, a graph description language that represents process structures as directed graphs using a concise syntax of nodes and edges, as an intermediate representation. In this pipeline, the paired process descriptions and DOT representations from the MaD dataset are used to fine-tune the Qwen2.5 14B Coder model, which is prompted to generate DOT Language outputs from the descriptions. This approach capitalizes on the reduced computational complexity of DOT, attributable to its generation of fewer tokens relative to BPMN, thereby enhancing the efficiency of creating these intermediate representations. The generation of DOT outputs in this pipeline is significantly faster than the direct BPMN generation in the Description-to-BPMN pipeline, with the Description-to-DOT pipeline being approximately 6 times faster for medium processes and 11 times faster for complex processes, as detailed in Sect. 4.2. Subsequently, a meticulously crafted Python script—the Graphviz-BPMN Converter, which, to the best of our knowledge, does not exist elsewhere—converts the generated DOT representation into BPMN 2.0 XML in milliseconds, producing a structured and standardized process depiction. This conversion also enables the generation of a DOT-to-BPMN dataset, which is used to further enrich the training data for the Description-to-BPMN pipeline. The Description-to-BPMN pipeline, by contrast, employs a fine-tuned Qwen2.5 14B Coder model to generate BPMN 2.0 XML directly from the MaD dataset's process descriptions, leveraging the generated DOT-to-BPMN dataset to improve model performance. While this pipeline bypasses intermediate representations like DOT during inference, it benefits from the enriched dataset during fine-tuning, albeit with higher token generation demands and slower completion times compared to the Description-to-DOT pipeline. Together, these pipelines integrate efficient generation techniques, model fine-tuning with the MaD dataset, and innovative scripting solutions to address the challenges of BPMN representation.

We present the following key contributions of our work:

- Limitations of LLMs: We identified significant limitations in LLMs for generating BPMN diagrams from textual descriptions, such as logical inconsistencies and hallucinations, highlighting the need for improved methods.
- Integrated Framework: To address these issues, we developed a framework that combines prompt engineering with fine-tuning of open-source SLMs on a BPM-specific dataset, resulting in more reliable and logically coherent BPMN outputs.

- Description-to-DOT Pipeline: We introduced a novel pipeline using a fine-tuned Qwen2.5 14B Coder model, which converts process descriptions to Graphviz DOT representations. This intermediate step reduces token generation compared to BPMN, enabling faster completion times—approximately 6 times faster for medium processes and 11 times faster for complex processes compared to direct BPMN generation.
- Benchmark Dataset: We created a new benchmark dataset for Description-to-BPMN tasks by converting MaD dataset pairs into standardized BPMN mappings, addressing a critical gap in available benchmarks for evaluating BPMN generation from text.
- Description-to-BPMN Pipeline: We developed and fine-tuned a pipeline that generates BPMN 2.0 XML directly from process descriptions, ensuring high accuracy and alignment with BPMN standards.
- Performance Validation: Our methods were rigorously tested on a wide range of process descriptions using various metrics, demonstrating robust performance, with consistently accurate BPMN outputs across varying complexity levels, as detailed in Sect. 4.2.

Our paper is organized as follows: Sect. 2 provides a detailed description of the dataset used for fine-tuning and benchmarking our work. Section 3 outlines our methodology, including the use of LLMs and the prompt engineering strategies tailored to the complexity of BPMN processes. We also discuss the rationale behind our choice of SLM and the algorithms used for converting Graphviz scripts to BPMN diagrams. Section 4 presents our results, including the performance evaluation of closed-source LLMs, a comparison with similar previous studies, and a comparative evaluation of description-to-BPMN methods across different complexity levels. Finally, Sect. 5 concludes the paper with a summary of our findings and future work.

#### Challenges of BPMN Generation using LLMs

While LLMs offer significant potential for generating BPMN XML outputs from textual process descriptions by leveraging their natural language understanding capabilities, this approach introduces several challenges. These challenges are particularly pronounced in the context of structured process modeling, where precision, adherence to standards, and graphical interpretability are paramount. We identify five key challenges that highlight the limitations of LLMs in the generation of BPMN.

- Lack of Explainability: LLMs can act as "black boxes" due to the complexity and size, making it difficult to comprehend the rationale behind specific outputs or decisions. As such, LLMs pose significant challenges for BPMN generation due to their lack of explainability [30] in generating specific BPMN elements, such as the placement of gateways or the alignment of sequence flows. Unlike traditional rule-based systems, which provide interpretable logic, LLMs do not offer mechanisms to trace the reasoning behind their generation of these elements. This lack of transparency is particularly detrimental in the BPMN domain, where users rely on process modeling tools like Camunda or bpmn.io to validate and refine generated diagrams. For instance, when an LLM incorrectly implements a parallel gateway or omits waypoints for edges, users cannot easily identify the underlying cause, which

undermines confidence in the generated outputs and hinders their practical adoption in business process management.

- Limited Output Control and Schema Adherence: LLMs are designed to process and generate plain text, making them well-suited for interpreting raw textual process descriptions as input. However, BPMN generation requires the output to conform to a specific XML schema (BPMN 2.0 standard), which includes structured elements such as process definitions (e.g., < bpmn: process>), sequence flows (e.g., < bpmn: sequenceFlow>), and graphical representations (e.g., < bpmndi: BPMNDiagram>). Instructing LLMs to adhere to this schema poses a significant challenge, as they often exhibit variability in their output, deviating from the required format. This limited control over generated output is particularly problematic in the BPMN domain, where precise distinctions — such as between sequential flows, parallel gateways (e.g., AND gateways), and exclusive gateways (e.g., XOR gateways) —are critical. Moreover, overlapping definitions, such as tasks versus subprocesses, can lead to structural inaccuracies, undermining the usability of the generated BPMN XML in modeling tools.
- Input Presentation and Complexity Dependencies: Although LLMs provide a natural language interface, the form, quantity, and level of detail in the input process descriptions must be carefully tailored to the complexity of the BPMN generation task. LLMs struggle to discern the importance of input components, particularly when process descriptions include complex elements such as parallel workflows, nested gateways, or conditional paths, which are challenging to describe concisely in natural language. Furthermore, LLMs' interpretation of inputs may diverge from human understanding, necessitating a trial-and-error approach to prompt optimization. This challenge is exacerbated in BPMN generation, where the input must convey both structural requirements (e.g., task sequences) and graphical requirements (e.g., layout alignment), often resulting in ambiguous or incomplete outputs that fail to meet BPMN standards.
- Domain-Specific Data Unawareness: LLMs are typically pretrained on general-purpose datasets, rendering them unaware of the specific characteristics of datasets used for BPMN generation, such as the MaD dataset employed in this study. This necessitates crafting precise instructions to describe the dataset's structure, conventions, and expected BPMN elements (e.g., gateways, tasks, sequence flows). However, the generalizing nature of LLMs can lead to the generation of irrelevant or incorrect BPMN elements based on their pretraining knowledge, particularly for complex process models involving nested or declarative workflows with diverse gateway types and constraints. This challenge requires additional effort to align the LLM's output with the specific requirements of the BPMN generation task, often through extensive prompt engineering or fine-tuning.
- Costly and Resource-Intensive Experiments: Conducting experiments with LLMs often involves commercial APIs (e.g., those for Mistral, GPT-4o, Gemini 1.5 Pro, or Claude 3.5 Sonnet), which impose constraints such as token limits on input and output sizes and fees based on token usage. In the context of BPMN generation, where process descriptions can be information-dense and the required BPMN XML outputs are detailed, the token requirements are substantial, leading to high computational and financial costs. This challenge is compounded by the need for

extensive prompt engineering, where multiple iterations of prompt design (e.g., Simple, Medium, Complex prompts) and variations in input complexity (e.g., Simple, Medium, Complex processes) are necessary to evaluate LLM performance comprehensively. Consequently, these experiments become resource-intensive and costly, posing a barrier to scalability and practical deployment in business process modeling.

These challenges highlight the inherent difficulties in using LLMs for BPMN generation, particularly in achieving the precision and structure required for process modeling. Addressing these challenges forms a core motivation for our study, as we propose fine-tuned methodologies using the Qwen2.5 14B Coder model to overcome these limitations, as detailed in Sect. 3.2 and validated in Sect. 4.2 and 4.3.

## Data description

We used a large-scale public benchmark dataset, the Business Process Model and Textual Description (MaD) dataset [15], which consists of 30,000 pairs of textual descriptions and corresponding BPMN diagrams in DOT format, spanning 15 distinct business domains, including accounts payable process, accounts receivable process, etc. The dataset was created through an iterative, quality-focused annotation process, where small batches were annotated, evaluated, and refined in cycles to ensure high consistency and reliability before scaling to the full dataset. It is divided into training (24,000 samples), validation (3,000 samples), and test sets (3,000 samples), which are further organized into 15 groups of business processes. Each group contains 1,600 samples for training, 200 for validation, and 200 for testing. This dataset is intended for research and development in process model automation, natural language understanding, and diagram generation. It can be used to train and evaluate models for Description-to-BPMN translation, process discovery, and process optimization tasks. Table 1 shows the number of steps and activities in each business domain.

Due to the scarcity of benchmark datasets for BPMN, this dataset addresses a critical gap in the field. It facilitates the automatic discovery and optimization of business process models across diverse business scenarios. Specifically, it supports the development

**Table 1** Business domains statistics of MaD dataset [15]

| Business domain | Steps | Activities |
|---|---|---|
| Accounts Payable Process (APP) | 4 | 17 |
| Accounts Receivable Process (ARP) | 5 | 13 |
| Budget Preparation Process (BPP) | 7 | 18 |
| Churn Rate Prevention Process (CRPP) | 8 | 21 |
| Client Onboarding Process for Marketing Agency (COPMA) | 4 | 21 |
| Content Promotion Process (CPP) | 2 | 12 |
| Customer Support Process for the Ticket Management (CSPTM) | 3 | 13 |
| Employee Onboarding Process (EOP) | 5 | 26 |
| Final Grades Submission Process (FGSP) | 5 | 19 |
| Loan Application Process (LAP) | 6 | 16 |
| Order Fulfillment Process (OFP) | 5 | 16 |
| Process for Optimizing a Process (POP) | 7 | 22 |
| Project Management Process (PMP) | 6 | 15 |
| Purchase Order Workflow (POW) | 5 | 11 |
| Startup Due Diligence for Venture Capitalist (SDDVC) | 6 | 33 |
| Total | **78** | **273** |

of large-scale pre-trained language models, such as BERT and GPT, which have recently achieved notable success and established a milestone in various practical applications. To enable the training and evaluation of models for Description-to-BPMN tasks, we employed the Graphviz-BPMN Converter, a custom Python script developed in this study, to transform the dataset's Graphviz DOT representations into BPMN 2.0 XML format. This conversion process generated a new dataset of 30,000 Description-to-DOT pairs, providing a standardized resource for fine-tuning and evaluating models that generate BPMN in DOT format directly from textual descriptions, thereby addressing the absence of such datasets in the field.

In this study, the dataset was categorized into three complexity levels - simple, medi um, and complex - with a distribution of 40% simple, 35% medium, and 25% complex as shown in Fig. 2. Each complexity level reflects distinct structural characteristics and process intricacies. Simple diagrams are defined by a constrained number of sequential steps, minimal decision points, and linear execution paths, representing straightforward processes with limited variability. Medium-complexity diagrams, in contrast, encapsulate more elaborate processes, characterized by an increased number of steps, the inclusion of branching pathways, and the presence of conditional and parallel flows, which introduce moderate levels of structural intricacy. Complex diagrams represent the highest level of sophistication, depicting highly detailed, multi-step operations with numerous decision points, extensive multi-level branching, and nested flow structures, thereby capturing intricate process dynamics with significant variability and interdependencies.

## Methodology

The methodology of this study is designed to address the challenges of generating BPMN outputs from textual process descriptions, comprising two primary components: an evaluation of LLMs to establish a baseline performance, and the development of pipelines using fine-tuned SLMs to overcome the identified limitations. This section is organized into three main subsections: the first details the evaluation of LLMs, including the prompt engineering strategies employed to assess their capabilities, the second describes the proposed pipelines leveraging a fine-tuned SLM, and the third presents the algorithms used for converting Graphviz scripts to BPMN diagrams.

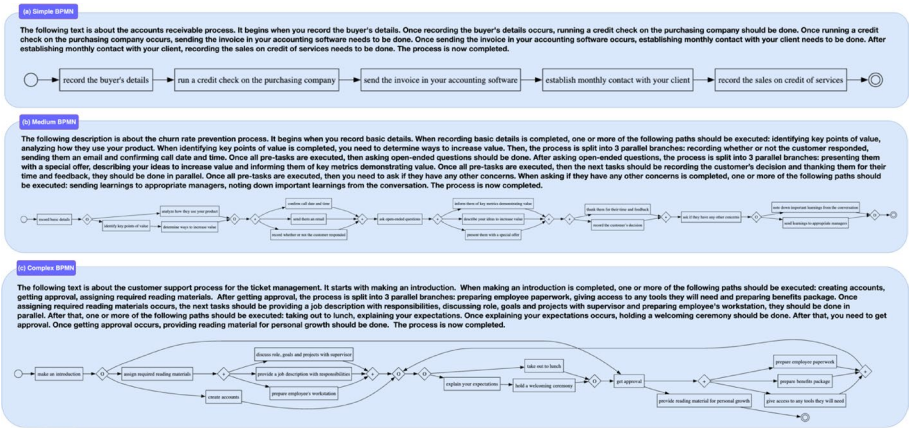



**Fig. 2** Examples of Complexity levels of Description-BPMN pair (**a**) Simple (**b**) Medium (**c**) Complex

### Large language models

LLMs represent a significant advancement in artificial intelligence, engineered to comprehend, generate, and manipulate human language with exceptional fluency and adaptability. Trained on extensive textual datasets, these models excel in diverse tasks, including question answering, text generation, coding, and logical reasoning. In our study, we applied Mistral, GPT-4o, Gemini 1.5 Pro, and Claude 3.5 Sonnet to generate BPMN 2.0 XML outputs, using standardized hyperparameters (temperature = 0.7, top_p = 1.0, max_tokens = 1,024) and consistent prompt formats for fair comparison. However, their application to domain-specific tasks, such as generating syntactically accurate BPMN diagrams, remains challenging due to their lack of specialized training in structured process modeling.

#### *Prompt engineering*

Prompt engineering is a critical technique in leveraging the capabilities of LLMs for specific tasks, enabling researchers to guide model behavior through carefully crafted instructions. In this study, prompt engineering strategies were employed to assess the capability of LLMs - Mistral, GPT-4o, Gemini 1.5 Pro, and Claude 3.5 Sonnet - to generate BPMN outputs through inference, focusing on their ability to interpret process descriptions and produce structured, processed outputs. Two primary approaches were explored: generating BPMN directly from textual process descriptions and generating BPMN from process descriptions paired with their corresponding DOT representations. These strategies, categorized into Simple, Medium, and Complex prompts, were designed to evaluate the LLMs' performance across varying levels of input complexity, providing insights into their strengths and limitations in structured process modeling. The strategies are detailed as follows:

**Simple prompt strategy: direct BPMN generation from descriptions** The Simple Prompt strategy focused on generating BPMN directly from textual process descriptions, targeting straightforward processes to establish a baseline for LLM performance. This strategy focuses on minimal instructions, aiming to produce a valid BPMN file with essential elements such as process definitions and sequence flow connections as shown in Fig. 3a. It served as a baseline to assess the LLMs' ability to interpret simple, linear, and unambiguous process descriptions without additional context or examples, testing their inherent understanding of BPMN conventions.

**Medium prompt strategy: BPMN generation from descriptions with corresponding DOT representations** The Medium Prompt strategy employed a one-shot learning approach, leveraging both the process description and its corresponding DOT representation as a single example to guide the LLMs in generating BPMN XML outputs for moderately complex processes incorporating guidelines to manage sequential and parallel flows as shown in Fig. 3b. This strategy leverages both the process description and its corresponding Graphviz DOT representation - a graph description language representing process structures as directed graphs - as a single paired example, enabling the LLM to infer the process structure.

By providing a single example of the process in both textual and DOT formats, this approach aligns with one-shot learning for structural understanding, as the LLM uses




**Fig. 3** Examples illustrating the varying complexity levels of BPMN prompts (**a**) Simple (**b**) Medium (**c**) Complex

the DOT representation to grasp the process structure (e.g., task sequences, parallel flows). However, since DOT is an intermediate representation and differs from the target BPMN XML format - which includes detailed semantics (e.g., gateway types) and graphical information (e.g., waypoints) - the LLM must also perform a zero-shot transformation to map the DOT structure to BPMN XML, relying on the prompt instructions and its pretraining to infer additional details. This hybrid approach enhances the LLMs' understanding by providing a structured graph-based reference alongside the textual description, improving the accuracy of the BPMN XML output. The strategy included explicit instructions for flow alignment (e.g., horizontal for sequential tasks, vertical for parallel tasks) and gateway handling (e.g., inclusive gateways for conditional paths), aiming to address the challenges of moderately complex processes with branching and parallel flows.

**Complex prompt strategy: enhanced BPMN generation from descriptions with corresponding DOT representations** The Complex Prompt strategy extended the one-shot learning approach to handle intricate process descriptions, incorporating a single

DOT representation as an example alongside comprehensive guidelines for generating detailed BPMN XML outputs. Additionally, this strategy employed role-based prompting by implicitly assigning the LLMs the role of a "BPMN modeler," expecting them to adhere to precise structural and graphical specifications. The prompt included detailed instructions for element arrangement, gateway types (e.g., exclusive (XOR), inclusive (OR), parallel (AND)), and graphical layout (e.g., spacing, waypoints), aiming to ensure both structural correctness and visual interpretability, as shown in Fig. 3c. This strategy was designed to test the LLMs' ability to manage complex processes with nested flows, multiple decision points, and intricate dependencies, providing a rigorous evaluation of their capabilities in structured process modeling. The combination of one-shot learning and role-based prompting aimed to enhance the LLMs' performance by providing both an example and a clear role context, encouraging adherence to BPMN standards.

Beyond zero-shot and one-shot learning, the prompt engineering strategies incorporated several additional techniques to optimize LLM performance for BPMN generation. Structured instructions were employed across all prompts to provide explicit guidance on BPMN requirements, such as the inclusion of graphical information (e.g., < bpmndi: BPMNDiagram > for layout) and sequence flow connections (e.g., <bpmn: sequenceFlow> with waypoints) [14, 29]. The Medium and Complex prompts further utilized constraint-based prompting by enforcing specific rules to ensure output consistency, such as "do not add names or descriptions for the gateways," which prevented the LLMs from generating extraneous metadata that could violate BPMN standards [14, 20, 29]. Additionally, the Complex prompt implicitly leveraged chain-of-thought prompting [12, 14, 19, 29] by encouraging the LLMs to reason step-by-step through the conversion of a Graphviz DOT representation into BPMN XML, considering intermediate steps like identifying gateway types (e.g., inclusive gateways for conditional paths), merging divergent paths, and adding waypoints to avoid overlaps in the graphical layout. While iterative refinement - such as multiple rounds of prompt adjustment based on feedback - was not employed in this study due to the inference-based evaluation setup, the progressive complexity of the prompts (from Simple to Medium to Complex) reflects a form of iterative design in the experimental framework [14, 21, 22]. This design systematically increased the challenge for the LLMs, enabling a comprehensive assessment of their capabilities across varying levels of process complexity.

These prompt engineering strategies were developed to systematically evaluate the performance of LLMs like Mistral, GPT-4o, and Gemini 1.5 Pro in BPMN generation. The Simple Prompt strategy tested their ability to generate BPMN directly from textual descriptions, while the Medium and Complex Prompt strategies assessed their capacity to produce BPMN from descriptions paired with corresponding DOT representations, providing a structured reference to enhance output accuracy. By varying the complexity of the prompts and inputs, this approach aimed to uncover the LLMs' limitations in generating logically consistent and structurally accurate BPMN outputs, particularly for complex and ambiguous inputs. These findings informed the development of fine-tuned approaches using the Qwen2.5 14B Coder model, as described in Sect. 3.2.

#### Small language models

SLMs are computational models designed to process and generate natural language with significantly fewer resources compared to their larger counterparts, such as well-known

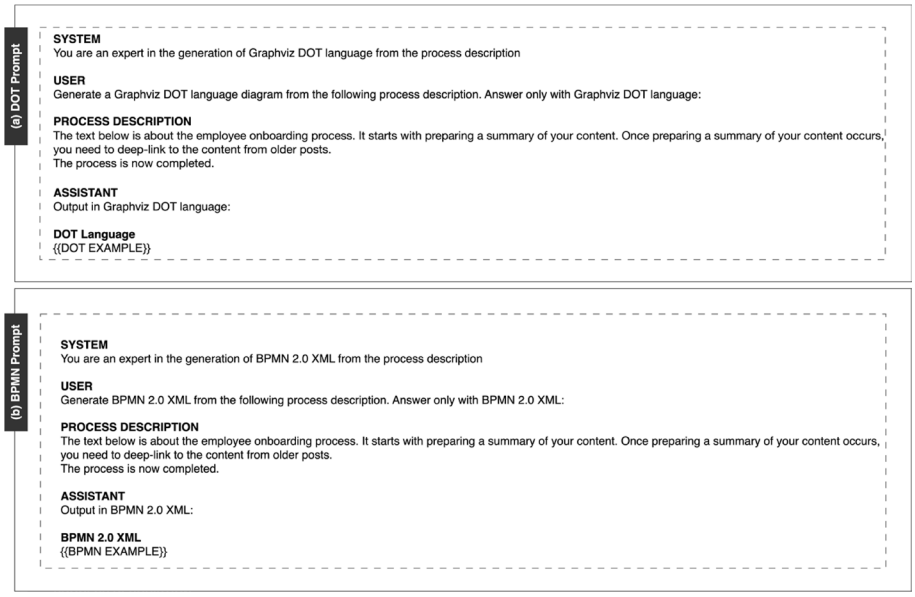



**Fig. 4** Examples of instruction prompts for training (**a**) Process description and DOT example pair (**b**) Process description and BPMN pair

LLMs [4, 6, 10, 17, 26, 28]. Despite their reduced size, SLMs can perform a wide range of NLP tasks while utilizing fewer parameters and simplified neural network architectures. This efficiency makes SLMs a practical alternative for specialized applications, offering functionality comparable to LLMs in resource-constrained environments. In the context of business process modeling, SLMs are particularly suitable for generating BPMN diagrams, as they can operate effectively with lower computational demands while maintaining the ability to produce structured and syntactically correct outputs.

### *Prompt engineering*

The generation of BPMN models from textual descriptions in this study was facilitated by a structured prompt engineering approach, designed to guide the language model in producing process models through two distinct pipelines: one generating Graphviz DOT language diagrams as an intermediate representation, and another directly generating BPMN models. The prompt design for the DOT pipeline, as exemplified by prompt as shown in Fig. 4a, was tailored to produce a structured intermediate representation that captures process flows for subsequent conversion to BPMN 2.0 XML, while the direct BPMN pipeline focuses on immediate BPMN output as depicted by prompt in Fig. 4b. Both pipelines employ a set of prompt engineering strategies to ensure accurate and structured outputs, which are detailed as follows:

- Role Assignment (Role-Based Prompting): The prompt defines the system as an expert in BPMN 2.0 XML generation, employing role-based prompting to establish a domain-specific focus. This encourages the model to apply specialized knowledge in process modeling, ensuring that both pipelines—whether outputting DOT as an intermediate step or directly generating BPMN—prioritize expertise in process representation. This technique has been shown to enhance task-specific performance in few-shot learning scenarios [1, 14].

- Task Specification with Constraints (Instruction-Based Prompting with Constraints): For the DOT pipeline, the prompt explicitly instructs the model to "answer only with Graphviz DOT language," using instruction-based prompting with constraints to prevent deviations into natural language or other formats. In the direct BPMN pipeline, a similar constraint requires the model to output BPMN 2.0 XML directly, ensuring the production of a machine-readable process model in the desired format for each pipeline. Instruction-based prompting with constraints is widely recognized for improving model adherence to task requirements [20, 29].
- Contextual Process Description (Contextual Prompting): Both pipelines utilize contextual prompting by providing a detailed description of the process to ground the task in a specific scenario. For the DOT pipeline, prompt (a) outlines the employee onboarding process with steps such as preparing a summary and deep-linking to content, facilitating accurate mapping to process components. The direct BPMN pipeline similarly provides a process description to enable precise identification of BPMN elements like start events, tasks, and gateways. Contextual prompting is effective in enhancing task understanding by supplying relevant background information [14, 19].
- Output Format Guidance (Format-Specification Prompting): In the DOT pipeline, the prompt includes a placeholder ("DOT Language<<DOT EXAMPLE>>") to cue the expected Graphviz DOT syntax, leveraging format-specification prompting to reinforce the structural requirements of the output. For the direct BPMN pipeline, the prompt specifies the BPMN 2.0 XML format as the target output, ensuring the model adheres to the standard syntax and structure of BPMN models in its responses. This technique aligns with few-shot prompting when examples are provided (e.g., a sample DOT structure), or zero-shot prompting when the model relies solely on the format specification, enabling it to generalize to new tasks without prior examples [1, 19]. Format-specification prompting is known to improve adherence to structured output requirements [7, 12].
- Focus on Structural Representation (Goal-Oriented Prompting): Both pipelines employ goal-oriented prompting by emphasizing the generation of outputs that capture the logical flow of the process. The DOT pipeline prioritizes a diagram that represents the sequence and dependencies of process steps for later conversion to BPMN, while the direct BPMN pipeline focuses on immediately producing a model with elements such as events, tasks, and gateways, ensuring the structural integrity of the process flow in both cases. Goal-oriented prompting aligns model outputs with specific objectives, such as producing structured process models [14, 32].

These prompt engineering strategies were designed to support the dual-pipeline approach, enabling the model to generate process models either through an intermediate DOT representation or directly as BPMN, depending on the pipeline employed. This methodology ensures flexibility and precision in transforming textual descriptions into structured process models.

#### *Qwen2.5 coder 14B*

Qwen2.5 Coder is a new series of language models engineered to deliver top-tier performance in coding tasks across a range of model sizes. Built upon the foundation of the Qwen2.5 LLMs, these models inherit an advanced architecture and tokenizer, optimized

for coding proficiency. The Qwen2.5 Coder series has been trained on extensive datasets and further refined through fine-tuning on meticulously curated instruction datasets tailored specifically for coding applications. With a mission to drive research and innovation in the realm of code-focused LLMs, coding agents, and coding assistant tools, the Qwen2.5 Coder series is released as a powerful, diverse, and practical solution to advance the development of open-source CodeLLMs. The series stands out in three key aspects: (1) Powerful: The Qwen2.5 Coder 32B Instruct model has emerged as the state-of-the-art (SOTA) open-source code model, rivaling the coding capabilities of GPT-4o while also excelling in general and mathematical skills. (2) Diverse: Offering six model sizes—0.5B, 1.5B, 3B, 7B, 14B, and 32B—Qwen2.5 Coder caters to a wide range of developer needs. (3) Practical: The series demonstrates real-world applicability in scenarios such as code assistants and Artifacts, with examples highlighting its potential to solve practical coding challenges [11].

Due to resource constraints, the Qwen2.5 Coder 14B model serves as the foundation for exploring its potential in BPMN generation. As part of the Qwen2.5 Coder series, the 14B model strikes a balance between computational efficiency and advanced capability, making it an ideal candidate for tasks requiring both precision and practicality. BPMN generation, which involves translating complex business processes into standardized visual workflows, aligns seamlessly with the strengths of Qwen2.5 Coder 14B. (1) Advanced Architecture Inherited from Qwen2.5 LLMs: The Qwen2.5 Coder 14B model is derived from the Qwen2.5 LLMs, which feature an advanced architecture optimized for understanding and generating complex, structured outputs. BPMN generation requires translating textual descriptions into a structured format with precise syntax (e.g., processes, gateways, and events). The robust architecture likely enables the model to handle such mappings effectively, leveraging its inherent ability to process and organize information hierarchically. (2) Extensive Training on Coding Datasets: Qwen2.5 Coder models are trained on extensive coding datasets, which include a variety of programming languages and structured formats. BPMN, while a notation standard, shares similarities with coding in its need for logical flow, conditionals, and sequential steps. This training likely equips the 14B model to interpret process descriptions as pseudo-algorithms and convert them into a BPMN-compatible structure, treating the task as an extension of its coding proficiency. (3) Fine-Tuning on Curated Instruction Datasets: Qwen2.5 Coder models are fine-tuned on carefully curated instruction datasets designed for coding tasks. If these datasets include examples of structured output generation or domain-specific tasks (e.g., workflow modeling), the model could generalize this knowledge to BPMN. Its ability to follow instructions like “Generate a BPMN diagram for a given process” suggests that the fine-tuning process enhanced its capacity to parse natural language inputs and produce rule-based, diagrammatic representations. (4) Strong General and Logical Reasoning Skills: The Qwen2.5 Coder series, including the 14B model, is noted for possessing good general and mathematical skills alongside coding capabilities. BPMN generation involves logical reasoning to identify start points, decision branches, and endpoints within a described process. The model’s reasoning abilities likely allow it to infer relationships and dependencies from descriptions (e.g., “If a condition is met, proceed to approval”) and translate them into BPMN elements like gateways and flows.

#### *Supervised fine-tuning (SFT)*

SFT is a widely adopted technique in the domain of machine learning, particularly for refining pre-trained LLMs to perform specialized tasks with improved accuracy and relevance. SFT involves further training a pre-trained model on a curated dataset comprising labeled input-output pairs, thereby aligning the model's behavior with specific task requirements. This process leverages the general knowledge encoded during the model's initial pre-training phase—typically conducted on vast, diverse corpora—and adapts it to a narrower, task-specific domain through supervised learning. The methodology entails defining a loss function, often based on cross-entropy or similar metrics, to minimize the discrepancy between the model's predictions and the ground-truth outputs provided in the training dataset.

In the context of LLMs, SFT is particularly valuable for enhancing the model's ability to follow instructions, generate structured outputs, or address domain-specific challenges. The training dataset typically consists of examples that mirror the intended application, such as question-answer pairs, instruction-response sequences, or descriptive prompts paired with expected outputs. Hyperparameters such as learning rate, batch size, and the number of training epochs are carefully tuned to balance adaptation and the preservation of pre-trained knowledge, mitigating risks like catastrophic forgetting—where the model loses previously learned capabilities. SFT's strength lies in its ability to bridge the gap between a model's broad, general-purpose proficiency and the precision demanded by targeted applications, making it a cornerstone of modern LLM optimization strategies.

#### *Parameter-efficient fine-tuning (PEFT)*

PEFT represents an advanced approach to adapting large-scale pre-trained models while minimizing computational overhead and resource demands. Unlike traditional fine-tuning, which updates all parameters of a model, PEFT focuses on modifying only a small subset of parameters or introducing lightweight, task-specific components, thereby preserving most of the pre-trained weights. This methodology is particularly advantageous for LLMs, where the sheer size of the parameter space—often numbering in the billions—renders full fine-tuning computationally expensive and memory-intensive. PEFT techniques, such as Low-Rank Adaptation (LoRA), adapters, or prompt tuning, enable efficient adaptation by targeting key architectural components, such as attention mechanisms or feed-forward layers, while maintaining the model's foundational knowledge.

PEFT is based on the hypothesis that task-specific adaptations can be effectively captured through low-dimensional updates to the model's parameter space, rather than requiring global retraining. For instance, LoRA introduces low-rank matrices to update weight matrices in transformer layers, significantly reducing the number of trainable parameters while achieving comparable performance to full fine-tuning. PEFT methods are typically configured with hyperparameters such as rank (in LoRA), scaling factors, or dropout rates, which are optimized to ensure effective learning within resource constraints. This approach not only enhances scalability but also facilitates rapid experimentation across multiple tasks, as the original model remains largely intact, allowing for reversible or modular adaptations. In practical settings, PEFT has emerged as a critical methodology for deploying LLMs in resource-limited environments, offering a balance between efficiency and task-specific performance [31].

#### *Hyperparameters tuning*

This section outlines the hyperparameter tuning strategy employed for Qwen2.5 Coder 14B, detailing the configurations used to optimize its performance for structured output generation tasks, such as Graphviz DOT or BPMN diagram generation. Table 2 presents the hyperparameters used for fine-tuning of Qwen2.5 Coder 14B. These parameters were selected to ensure effective adaptation of the model within resource constraints, targeting specific architectural components to enhance task-specific performance.

The Qwen2.5 14B Instruct model [25]was fine-tuned on two distinct datasets—30,000 description-DOT pairs and 30,000 description-BPMN pairs—using an NVIDIA RTX 6000 GPU, with one epoch proving sufficient to avoid overfitting [24]. These datasets enabled the model to independently learn mappings from natural language process descriptions to Graphviz DOT language and BPMN notations, enhancing its versatility in generating structured workflow representations across both formats. Table 2 presents the hyperparameter settings applied during fine-tuning to ensure effective adaptation within the GPU's resource constraints.

### Graphviz to BPMN conversion

The transformation of a process model from a DOT representation to a BPMN-compliant diagram is achieved through a sequence of interconnected algorithms, each addressing a distinct aspect of the conversion process: structural extraction, spatial alignment, logical mapping, and diagrammatic representation. Together, these algorithms ensure that the resulting BPMN model accurately captures both the semantics and visual clarity of the original process encoded in DOT format.

The process begins with Algorithm 1: Build Directed Graph, which serves as the foundation by parsing the DOT content (e.g., a string gv_content) and constructing a directed graph $G=(V, E)$ using a library like NetworkX (nx.DiGraph). This step extracts nodes (representing process elements) and edges (representing flow dependencies) while preserving attributes such as labels and shapes. By recursively processing subgraphs, Algorithm 1 ensures that even complex, nested structures in the DOT input are fully represented in $G$, providing a robust structural backbone for subsequent steps.

Next, Algorithm 2: Build Flow Alignment takes the directed graph $G$ as input and computes a spatial layout $L$, mapping each node to coordinates in a 2D plane. This algorithm performs topological sorting—or an alternative sorting method if cycles are

**Table 2** Qwen2.5 coder 14B hyperparameters fine-tuning

| | Hyperparameter | Value |
|---|---|---|
| | Epoch | 1 |
| | Learning rate | 2e-4 |
| | Learning rate scheduler | Linear |
| | Batch size | 2 |
| | Gradient accumulation steps | 4 |
| | Warm up steps | 5 |
| | Weight decay | 0.01 |
| Fine-tuning | Modules | Attention layers (Query, Key, Value, and Output projectors) and feed-forward layers (gate, up, and down) |

present—to assign hierarchical levels to nodes, reflecting their execution order. It then calculates node positions by grouping nodes by level, applying spacing rules (e.g., x_spacing, y_spacing), and adjusting for layout challenges such as backward edges (loops) or overlapping elements. The spatial layout output L ensures that the BPMN diagram will be visually organized, with gateways and other elements aligned to enhance readability while respecting the process flow defined in *G*.

With the structure and layout established, Algorithm 3: Create BPMN Process translates the abstract graph *G* into a logical BPMN model *P*. This step maps each node to its corresponding BPMN element—start events for START_NODE, end events for END_NODE, gateways for decision points, and tasks for activities—while linking them with sequence flows based on the edges in *E*. By maintaining a mapping of element IDs, Algorithm 3 constructs a process model that adheres to BPMN semantics, ensuring that the execution logic of the original DOT representation is preserved in a standardized format.

Finally, Algorithm 4: Create BPMN Diagram integrates the logical model P from Algorithm 3 with the spatial coordinates *C* from Algorithm 2 to produce a complete BPMN diagram *D* in XML format, compliant with BPMN 2.0 standards. For each node, it creates a BPMNShape element positioned according to *C(v)*, and for each edge, it generates a BPMNEdge with waypoints derived from the source and target coordinates. This step bridges the logical and visual aspects, resulting in a diagram that is both semantically accurate and visually structured, suitable for tools like BPMN modelers or process analysis software.

In summary, the algorithms are tightly linked: Algorithm 1 extracts the raw structure from DOT, Algorithm 2 arranges it spatially, Algorithm 3 maps it to BPMN's logical framework, and Algorithm 4 renders it as a fully realized diagram. This multi-stage approach ensures a seamless transformation, preserving the process's integrity from a text-based DOT input to a standardized, visually intuitive BPMN output.

| | |
|---|---|
| 1: **Input:** | $gv_content$: A string containing DOT content |
| 2: **Output** | $\boldsymbol{G} = (\mathbf{V}, \mathbf{E})$: A directed graph ($nx.DiGraph$) representing the process flow. |
| 3: | Parse gv_content using pydot.graph_from_dot_data |
| 4: | Initialize an empty directed graph $G$ |
| 5: | **For** each $node \in V$ do |
| 6: | Extract node_id, label, and shape |
| 7: | Add node to G with attributes |
| 8: | **For** each edge ∈ E do |
| 9: | Extract source and target |
| 10: | Add edge to $G$, including labels if present |
| 11: | **For** each subgraph in $gv_content$ do |
| 12: | Recursively process subgraph |
| 13: | **Return** $G$ |

**Algorithm 1** Build directed graph

**Input:** $G = (V, E)$: A directed graph ($nx.DiGraph$) representing the process flow.
**Output** $L$: Node positions mapping each node to its coordinates
Initialize $node_coords = \{\}$
Try topological sorting on $G$
    **if successful** assign $sorted_nodes \leftarrow topological_sort(G)$
    **else** compute $sorted_nodes \leftarrow alternative_sort(G)$
Set spacing values x_spacing, y_spacing, and initial positions base_x, base_y
**For** each node ∈ sorted_nodes do
    **if** node has no predecessors then $levels[node] \leftarrow 0$
    **else** $levels[node] \leftarrow max(levels[p] \forall p \in predecessors(node)) + 1$
**if** $END_NODE \in V$ *then* $levels[END_NODE] \leftarrow max(levels) + 1$
Group nodes by level into nodes_by_level
**For** each level ∈ [0, max(levels)] do
    Determine vertical spacing for nodes in level
    Compute $(x, y)$ positions with adjusted alignment for gateways
Adjust for backward edges (loops) by shifting affected nodes
Compute alternative adjustments using $AlternativeOverlapResolution(G, L)$
**Return** $L$

**Algorithm 2** Build flow alignment

**Input:** $G = (V, E)$: A directed graph ($nx.DiGraph$) representing the process flow.
**Output** **P:** A PMN process element containing nodes and sequence flows
**Initialize** BPMN process $P$
Create mapping $element_ids = \{\}$ to track BPMN elements
**For** each node **∈ V** do
    **if** $node == START_NODE$ **then** create startEvent
    **else if** $node == END_NODE$ then create endEvent
    **else if** node is a gateway **then** create appropriate gateway
    **else if** node is a gateway **then** create appropriate gateway
    **else if** node is an activity **then** create task
**For each** $(source, target) \in E$ **do**
    Create sequenceFlow linking source to target
**Return** $P$

**Algorithm 3** Create BPMN process

**Input:** $G = (V, E)$**:** A directed graph ($nx.DiGraph$) representing the process flow.
$C: V \rightarrow R^2$**:** A dictionary mapping each node in $G$ to its corresponding coordinates.
**Output:** $D$: BPMN diagram in XML format, structured according to BPMN 2.0 standards.
Initialize Diagram Elements
    Create root element $D$ as BPMNDiagram.
    Add BPMNPlane and link it to Process_1.
**For** each node $v \in V$:
    Retrieve coordinates $(x, y)$ from $C(v)$.
    Assign a unique BPMN ID based on node type
    Create corresponding BPMNShape with position derived from $C(v)$.
**For** each edge $(u, v) \in E$:
    Assign a unique Flow_ID.
    Retrieve coordinates $(sourcex, sourcey)$ for $u$ and $(targetx, targety)$ for $v$.
    Determine waypoints for BPMNEdge
    Create a BPMNEdge element with the calculated waypoints.
**Return** $D$

**Algorithm 4** Create BPMN diagram

## Results

This section presents the findings from our evaluation of various approaches to generating BPMN models from textual descriptions. The results are organized into three subsections: an assessment of closed-source LLMs, a comparison with prior studies, and a comparative evaluation of our approach against existing methods across different complexity levels.

### Performance evaluation of closed-source LLMs

The performance of LLMs —Mistral, GPT-4o, Gemini 1.5 Pro, and Claude 3.5 Sonnet— in generating BPMN outputs was evaluated across Simple, Medium, and Complex process descriptions, as summarized in Table 3. The evaluation criteria, encompassing structural accuracy, graphical representation, flow control, and compliance with the BPMN 2.0 standard, were tailored to the complexity of each process description. These criteria are defined as follows:

**Table 3** Performance comparison of LLMs in BPMN generation across complexity levels

| Complexity | Criterion | Mistral | GPT-4o | Gemini 1.5 Pro | Claude 3.5 Sonnet |
|---|---|---|---|---|---|
| Simple | Process structure definition | Yes | Yes | Yes | Yes |
| | Graphical representation | Yes | Yes | Yes | Yes |
| | BPMN standard compliance | Yes | Yes | Yes | Yes |
| Medium | Process structure definition | Yes | Yes | Yes | Yes |
| | Graphical representation | Yes | Yes | No | No |
| | Gateway logic accuracy | Yes | Yes | Yes | No |
| | Gateway merging completeness | Yes | Yes | Yes | No |
| | Waypoint inclusion for edges | Yes | Yes | No | No |
| | BPMN standard compliance | Yes | Yes | Yes | No |
| Complex | Process structure definition | Yes | Yes | Yes | Yes |
| | Graphical representation | No | No | No | No |
| | Flow alignment accuracy | No | No | No | No |
| | Gateway logic accuracy | Yes | No | No | No |
| | Gateway merging completeness | Yes | No | No | No |
| | Waypoint inclusion for edges | No | No | No | No |
| | Element connectivity | No | No | No | No |
| | Element spacing and readability | No | No | No | No |
| | BPMN standard compliance | Yes | Yes | No | No |

- Process Structure Definition: Assesses whether the BPMN includes a complete and accurate process definition (<bpmn: process>), capturing all tasks and elements.
- Graphical Representation: Evaluates the presence and correctness of graphical elements (<bpmndi: BPMNDiagram>), including shapes and edges.
- Flow Alignment Accuracy: Measures the correct alignment of sequential tasks (horizontal) and parallel/alternative paths (vertical) in the graphical layout.
- Gateway Logic Accuracy: Assesses the correct implementation of gateways (exclusive, inclusive, parallel) and their logic in the process flow.
- Gateway Merging Completeness: Evaluates whether split gateways are properly closed with corresponding merging gateways.
- Waypoint Inclusion for Edges: Checks for the inclusion of waypoints (<bpmndi: BPMNEdge>) to prevent overlapping in sequence flows.
- Element Connectivity: Ensures proper connectivity between elements (<bpmn: sequenceFlow>), reflecting the process description accurately.
- Element Spacing and Readability: Assesses whether elements are spaced adequately to avoid overlaps and enhance visual clarity.
- BPMN Standard Compliance: Verifies if the generated BPMN adheres to the BPMN 2.0 standard and can be parsed by a modeler tool.

For Simple processes, all models —Mistral, GPT-4o, Gemini 1.5 Pro, and Claude 3.5 Sonnet— successfully generated BPMN outputs with accurate process structure definitions (e.g., < bpmn: process > elements), graphical representations (e.g., < bpmndi: BPMNDiagram > for layout), and compliance with BPMN 2.0 standards, demonstrating their capability to handle linear and unambiguous inputs effectively. However, as process complexity increased, their performance deteriorated significantly. For Medium-complexity processes, Mistral and GPT-4o consistently produced BPMN that met all specified criteria, including process structure, gateway logic, flow alignment, waypoint inclusion, and element spacing. In contrast, Gemini 1.5 Pro maintained accuracy in process structure and gateway logic but struggled with flow alignment, waypoint inclusion,

and element spacing, leading to outputs that were visually misaligned and difficult to parse in modeling tools. Claude 3.5 Sonnet, however, was limited to generating only the process structure definition, failing to incorporate graphical elements or adhere to layout requirements. For Complex processes, the challenges were even more pronounced: while all models achieved basic process structure definitions, they consistently failed to address flow alignment, gateway merging, waypoint inclusion, element connectivity, and spacing. Gemini 1.5 Pro and Claude 3.5 Sonnet exhibited the most significant deficiencies across all criteria, producing outputs that were structurally incomplete and graphically unusable. These findings highlight the inherent limitations of LLMs in generating BPMN XML for intricate processes, particularly in the absence of domain-specific training, and underscore the necessity of the proposed fine-tuned methodologies, such as the Description-to-DOT and Description-to-BPMN pipelines, to overcome these shortcomings.

#### Comparison with previous studies

This section compares prior work on generating BPMN models from textual descriptions using LLMs, focusing on approaches by [13, 16, 18], alongside our work. As summarized in Table 4, these studies predominantly rely on closed-source LLMs (e.g., GPT-4o, Claude 3.5 Sonnet, Gemini variants), introducing significant costs and accessibility challenges due to their proprietary nature, though our work explores open-source alternatives like Qwen2.5 Coder 14B model, offering a cost-effective alternative.

Kourani et al. [13] introduce ProMoAI, which uses a Partially Ordered Workflow Language (POWL) as an intermediate representation, leveraging Python code generation via LLMs (e.g., Claude 3.5 Sonnet, GPT-4o, Llama 3.1) with advanced prompt engineering to ensure sound models convertible to BPMN or Petri nets. Evaluated across 16 state-of-the-art LLMs using a dataset of 20 diverse business processes, finding significant performance variations where Claude-3.5-Sonnet achieved the highest average quality score of 0.93, with a positive correlation between effective error handling and higher quality, and time efficiencies led by models like Gemini-1.5-Flash-002 at 14.51 s average total time. The self-improvement analysis demonstrated that techniques such as self-evaluation, input optimization, and output optimization yielded modest average gains (up to 0.84 in specific cases), with output optimization proving particularly promising for enhancing weaker LLMs,, though limited to control-flow perspectives and challenged by complex descriptions.

Nour Eldin [18] present Nala2BPMN, a hybrid approach that automates BPMN generation by refining textual descriptions via prompt engineering, extracting entities (e.g., activities, events) and relationships (e.g., dependencies, branching), and using a structured algorithm to construct BPMN models, with user validation to minimize errors. Evaluated on a subset of 8 process descriptions from the PET dataset (47 description-BPMN pairs) [5], Nala2BPMN outperformed baselines like ProMoAI using GPT-4o and GPT-4-turbo, achieving a 95% success rate in generating executable models, a 40% reduction in token usage compared to ProMoAI, and up to 70% reduction in manual effort. Qualitative assessments by 12 BPM professionals and academics yielded average scores of 4.5/5 for understandability, 4.6/5 for correctness, 4.4/5 for completeness, 4.5/5 for relevance, and 4.7/5 for ease of updating, compared to ProMoAI's scores of 3.2, 3.1, 3.0, 3.3, and 3.0, respectively, with higher F1-scores for entity/relation extraction and

**Table 4** Summary of previous studies on description-to-BPMN

| Authors, year | Intermediate representation | Dataset | Dataset complexity | Models | Evaluation metrics | Metric results | Latency | Cost |
|---|---|---|---|---|---|---|---|---|
| [13] | Partially Ordered Workflow Language (POWL) | Custom (20 processes) | Simple to complex (8–26 activities) | Claude 3.5 Sonnet, GPT-4o, Llama 3.1, others (16 LLMs) | Quality score, error-handling iterations, generation time | Peak quality 0.93 (Claude 3.5 Sonnet) | 14.51s (Gemini-1.5-Flash-002) | - |
| [16] | JSON | Custom (60 generation, 40 editing tasks) | Simple to moderate | GPT-4o, Claude 3.5 Sonnet, o3-mini, others (8 LLMs) | Similarity (GED), success rate, latency, failures | 68% editing success (JSON) vs. 65% (XML), Similarity 0.70 vs. 0.69 (XML) | 14.41s vs. 24.07s (generation), 21.46s vs. 46.98s (editing) | 2 vs. 12 failures (resource efficiency) |
| [18] | Directed graph | PET (47 pairs, 8 tested) | Simple to complex | GPT-4o, GPT-4-turbo | F1-score, token usage, expert ratings (1–5 scale) | 95% success rate, Ratings 4.4–4.7/5 | - | 40% token reduction, 70% manual effort reduction |
| Our work | DOT language | MaD (30,000 pairs) | Simple, medium, complex | Qwen2.5 Coder 14B | Accuracy, line count, generation time | 97.3% (DOT), 88% (BPMN) | 2.5m vs. 15m (medium), 3.5m vs. 40m (complex) | 4.4-4.5x fewer lines (55 vs. 241, 77 vs. 341), 6x faster (medium), 11x faster (complex) |

lower logical errors, though challenges persisted with complex loops and occasional hallucinations due to reliance on closed-source LLMs.

Licardo [16] present BPMN Assistant, a tool that automates BPMN generation from textual descriptions by leveraging LLMs to produce JSON representations of elements (e.g., tasks, gateways, events), which are converted to BPMN XML with automatic layout coordinates using the bpmn-auto-layout library, and supports iterative editing via natural language instructions invoking functions like add_element or redirect_branch. Evaluated on 60 BPMN generation tasks across eight LLMs (e.g., GPT-4o mini, GPT-4o, Claude 3.5 Sonnet, o3-mini, Gemini 1.5 Flash, Llama 3.70B Instruct, Qwen 2.5 72B Instruct, DeepSeek V3), JSON outperformed XML with an average similarity score of

0.70 (vs. 0.69 for XML) and fewer failures (2 vs. 12), with Claude 3.5 Sonnet and o3-mini showing particularly strong performance; JSON also demonstrated efficiency with a mean latency of 14.41 s (vs. 24.07s for XML), lower output tokens (849.29 vs. 1834.55), and higher input tokens (2604.34 vs. 466.79), reflecting a trade-off of richer prompts for reduced processing time. For 40 editing tasks, JSON achieved up to 68% success rate (e.g., Claude 3.5 Sonnet) compared to 65% for XML, with lower latency (21.46s vs. 46.98s), enhancing reliability for non-technical users through iterative refinements, though performance varied significantly across LLMs, suggesting a need for further investigation into model-specific strengths. However, the tool is limited to a subset of BPMN elements (excluding pools and lanes), struggles with ambiguous inputs, and lacks semantic evaluation or usability studies.

Our approach automates BPMN generation by employing the DOT language as a graph-based intermediate representation, leveraging a fine-tuned version of the open-source Qwen2.5 Coder 14B model to transform textual descriptions into structured process models that are subsequently converted to BPMN. The DOT language, derived from Graphviz, represents processes as directed graphs with nodes (e.g., tasks, events) and edges (e.g., sequence flows), enabling a modular and lightweight representation that simplifies complexity management before BPMN synthesis. Evaluated on the large-scale MaD dataset (30,000 description-BPMN pairs), which encompasses a wide range of simple, medium, and complex processes across domains like logistics and finance, this method addresses the cost and accessibility barriers posed by closed-source LLMs (e.g., GPT-4o, Claude 3.5 Sonnet) used by [13, 16, 18]. The DOT-to-BPMN pipeline achieves a 97.3% accuracy for DOT representations and 88% for final BPMN models, validated through graph isomorphism between the predicted graph (G1) and reference graph (G2), ensuring structural fidelity across the extensive dataset—outperforming the accuracy metrics of Kourani et al. (0.93 peak quality), Licardo et al. (68% editing success), and Nour Eldin et al. (95% success) while surpassing the scale of their datasets (20, 100, and 47 pairs, respectively). Performance metrics reveal a trade-off: for medium-complexity processes as shown in Table 5, DOT generates 55 lines of code in 2.5 min compared to BPMN's 241 lines in 15 min, and for complex processes, 77 lines in 3.5 min versus 341 lines in 40 min, measured on an NVIDIA RTX 6000 GPU. This translates to a 4.4–4.5x reduction in code lines and significant token efficiency, though the higher latency (2.5–3.5 min) compared to the sub-minute latencies of closed-source models (e.g., 14.51s for Kourani et al.) is attributed to the open-source Qwen2.5 Coder 14B's focus on robust accuracy and resource optimization over raw speed, as illustrated in Fig. 5. The efficiency gains are further highlighted in Fig. 6, which shows DOT's advantage in generating fewer lines and completing processes faster than BPMN. Unlike Licardo et al.'s JSON reuse, which iteratively refines a JSON structure for editing, our DOT optimization preprocesses complexity by abstracting process logic into a graph format, minimizing LLM

**Table 5** Summary of performance metrics of Description-to-DOT and Description-to-BPMN pipelines

| Metric | DOT (Medium) | DOT (Complex) | BPMN (Medium) | BPMN (Complex) |
|---|---|---|---|---|
| No. of Lines | ~55 | ~77 | ~241 | ~341 |
| Average generation Completion (Minutes) | 2.5 | 3.5 | 15 | 40 |
| Accuracy | 97.3% (DOT) | | 88% (BPMN) | |

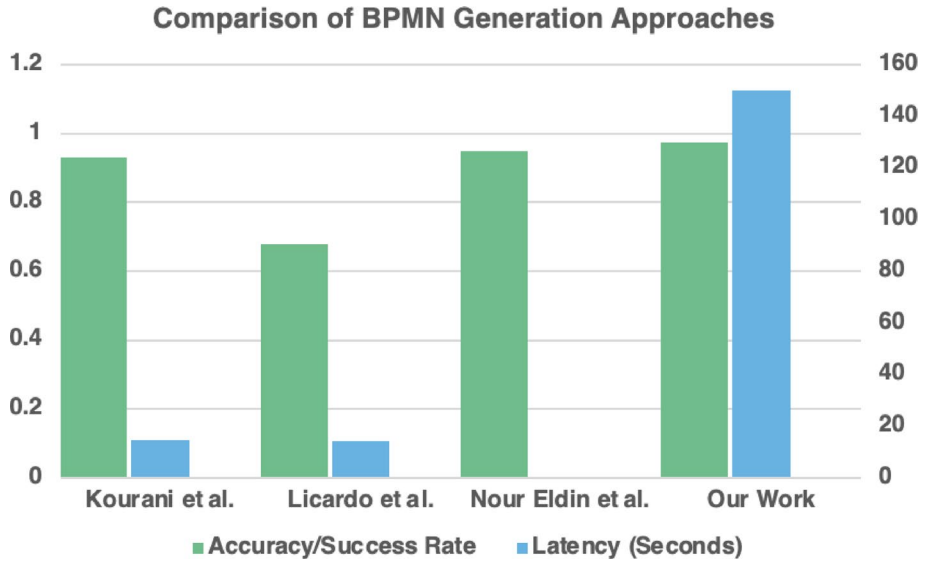



**Fig. 5** Comparison of BPMN generation approaches: accuracy/success rate and latency (seconds)

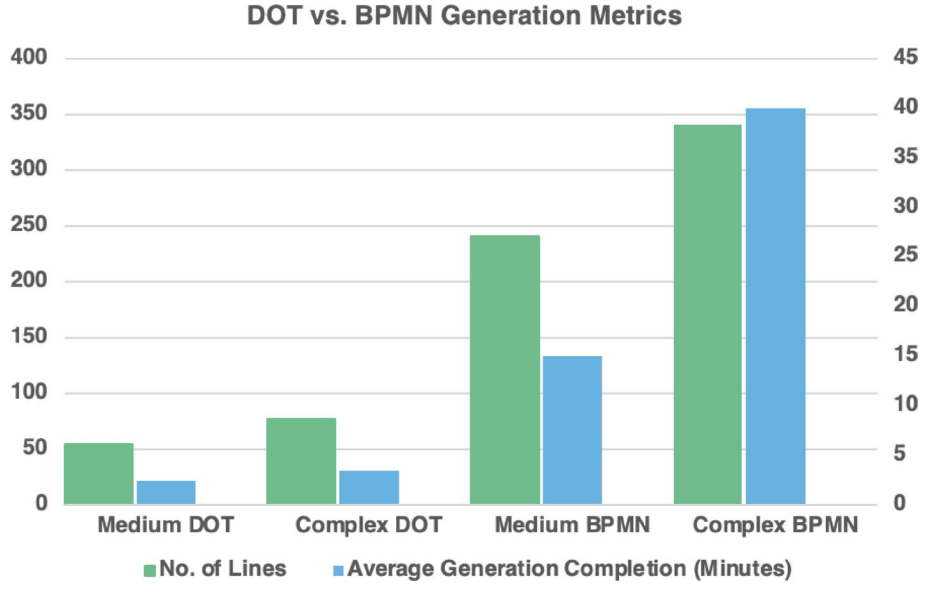



**Fig. 6** DOT vs. BPMN generation metrics: number of lines and average generation completion (minutes)

processing load and enabling rapid conversion to BPMN despite the latency trade-off. This scalable, cost-effective alternative leverages open-source technology to democratize access, maintaining superior accuracy and adaptability for large-scale process modeling in terms of complexity.

#### Comparative evaluation of description-to-BPMN methods across complexity levels

The performance of two established methods for generating BPMN models from textual descriptions, ProMoAI [13] and BPMN Assistant [16], was evaluated across Simple, Medium, and Complex process descriptions, as summarized in Table 6. ProMoAI utilizes the Llama 3.1 model with advanced prompt engineering to generate models via a Partially Ordered Workflow Language (POWL) intermediate representation, while

**Table 6** Comparison of BPMN generation methods across complexity levels: ProMoAI, BPMN assistant, and our approach

| Complexity | Criterion | ProMoAI [13] | BPMN Assistant [16] | Our work |
|---|---|---|---|---|
| Simple | Process structure definition | Yes | Yes | Yes |
| | Graphical representation | Yes | Yes | Yes |
| | BPMN standard compliance | Yes | Yes | Yes |
| Medium | Process structure definition | No | No | Yes |
| | Graphical representation | Yes | Yes | Yes |
| | Gateway logic accuracy | No | No | Yes |
| | Gateway merging completeness | Yes | Yes | Yes |
| | Waypoint inclusion for edges | Yes | Yes | Yes |
| | BPMN standard compliance | Yes | Yes | Yes |
| Complex | Process structure definition | No | No | Yes |
| | Graphical representation | Yes | Yes | Yes |
| | Flow alignment accuracy | Yes | Yes | Yes |
| | Gateway logic accuracy | No | No | Yes |
| | Gateway merging completeness | Yes | Yes | Yes |
| | Waypoint inclusion for edges | Yes | Yes | Yes |
| | Element connectivity | Yes | Yes | Yes |
| | Element spacing and readability | Yes | Yes | Yes |
| | BPMN standard compliance | Yes | Yes | Yes |

BPMN Assistant employs the DeepSeek V3 model to produce JSON representations convertible to BPMN XML. The evaluation employed criteria including process structure definition, graphical representation, gateway logic accuracy, and BPMN standard compliance, with additional metrics for Medium and Complex levels such as flow alignment accuracy, gateway merging completeness, waypoint inclusion, element connectivity, spacing, and readability.

The results reveal distinct performance patterns across complexity levels. For Simple processes, both ProMoAI and BPMN Assistant successfully met all criteria, including process structure definition, graphical representation, and BPMN standard compliance. However, as complexity increased to Medium and Complex levels, both methods exhibited shortcomings, failing to achieve accuracy in process structure definition and gateway logic, despite meeting the remaining criteria (e.g., graphical representation, flow alignment, waypoint inclusion). This suggests that while these methods are effective for basic process modeling, they struggle with the intricacies of more complex workflows, particularly in defining accurate process structures and gateway logic. In contrast, our DOT-based approach with the fine-tuned open-source Qwen2.5 Coder 14B model excels in generating complex BPMN, delivering robust process structure definition and gateway logic across all complexity levels, validated on the MaD dataset (30,000 pairs).

## BPMN modeler frontend

The BPMN Modeler frontend is a web interface that provides an intuitive graphical user experience, allowing users to convert text descriptions into BPMN without installing external dependencies. Powered by a fine-tuned, open-source Qwen2.5 Coder 14B model, the application runs internally, eliminating the need for API keys and ensuring privacy and control within the organization.

As shown in Fig. 7, the interface enables users to select output formats (Graphviz or BPMN XML), input process descriptions, and efficiently visualize the resulting diagram.

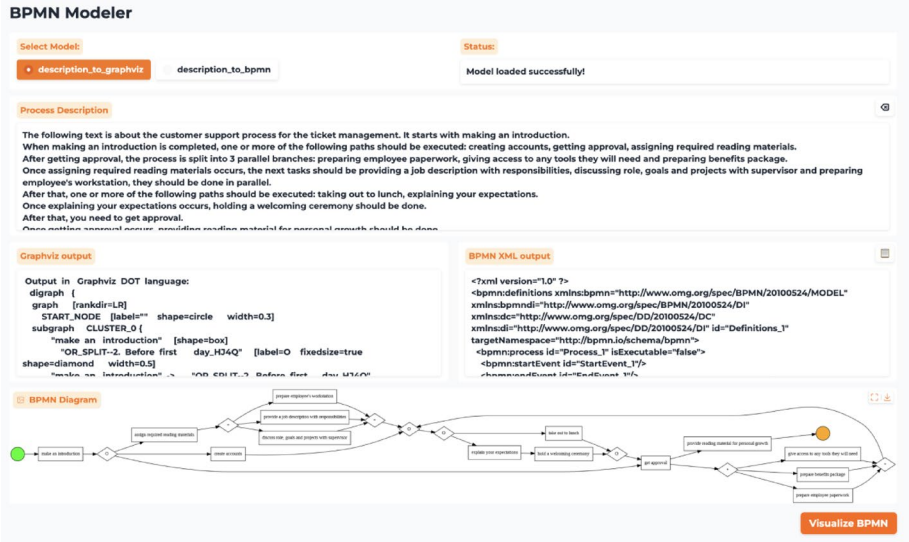



**Fig. 7** Screenshot of BPMN process generator application

Users can also copy BPMN diagrams in standard formats for easy integration with downstream tools. Its modular approach allows users to handle processes of varying complexity, optimizing both accuracy and resource usage without relying on external dependencies.

## Conclusion

This study has advanced the field of automated business process modeling by presenting an efficient approach for generating BPMN XML outputs from textual process descriptions, utilizing the open-source Qwen2.5 Coder 14B model and evaluated on the MaD dataset. Through a comprehensive evaluation of LLMs—Mistral, GPT-4o, Gemini 1.5 Pro, and Claude 3.5 Sonnet—we identified key limitations, such as limited output control, input presentation dependencies, and a lack of explainability, particularly for complex processes with nested flows and intricate dependencies.

To address these challenges, we proposed a novel Description-to-DOT pipeline, leveraging Graphviz DOT as an intermediate representation to reduce token generation demands, followed by a Python script that converts DOT to BPMN XML in milliseconds. As demonstrated in Tables 4 and 5, the Description-to-DOT pipeline, leveraging the fine-tuned Qwen2.5 Coder 14B model, offers substantial efficiency improvements over the direct Description-to-BPMN pipeline, achieving 4.4–4.5x fewer lines (55 vs. 241 for medium, 77 vs. 341 for complex) and accelerating generation times by 6x for medium and 11x for complex processes, as measured on an NVIDIA RTX 6000 GPU. This approach ensures robust performance across all complexity levels, with the fine-tuned model delivering accurate structural integrity through the DOT representation, validated via graph isomorphism to confirm structural equivalence between predicted and reference models on the MaD dataset (30,000 pairs), making it a versatile and effective solution for generating complex BPMN models.

Additionally, we contributed a new benchmark dataset for Description-to-BPMN tasks, derived from the MaD dataset using the Graphviz-BPMN Converter, providing a standardized resource for evaluating models that generate BPMN directly from textual

descriptions. These findings highlight the potential of fine-tuned SLMs in overcoming the challenges of LLMs for structured process modeling tasks, offering a cost-effective, efficient, and scalable solution.

Future work can build on this foundation in several directions. First, enhancing the explainability of the fine-tuned model by developing methods to trace the reasoning behind generated BPMN elements, such as through intermediate reasoning steps or visualization of token generation processes, could further improve user confidence and adoption in business process management. Second, the benchmark dataset can be expanded to include more diverse process types, such as swimlanes to support more comprehensive evaluations. Finally, extending the evaluation framework to incorporate user studies or real-world deployment scenarios could provide deeper insights into the practical utility of the generated BPMN models, ensuring their applicability across diverse business process modeling contexts. These directions aim to advance the scalability, transparency, and practical impact of automated BPMN generation systems.

## Appendix

A Prompt Templates

A.1 Medium and Complex Prompts for LLM

(b) Medium Prompt

**INSTRUCTIONS**

Convert the following BPMN diagram gv code into xml format. The response should be a valid BPMN xml file and should be parsable by its modeler tool. There should be nothing except the xml in the response. Ensure that the graphical information with the connections on sequence flows are included. Adjust the alignment of the blocks so the sequential blocks go left to right and the parallel ones to be aligned vertically. "one or more of the following paths should be executed:" Means inclusive gateway. whenever you add a gateway, add another one to merge the paths and Add waypoints to the BPMNEdges to avoid overlapping with blocks if needed. Do not add names or descriptions for the gateways

**PROCESS DESCRIPTION**

The following description is about the churn rate prevention process. It begins when you record basic details. When recording basic details is completed, one or more of the following paths should be executed: identifying key points of value, analyzing how they use your product. When identifying key points of value is completed, you need to determine ways to increase value. Then, the process is split into 3 parallel branches: recording whether or not the customer responded, sending them an email and confirming call date and time. Once all pre-tasks are executed, then asking open-ended questions should be done. After asking open-ended questions, the process is split into 3 parallel branches: presenting them with a special offer, describing your ideas to increase value and informing them of key metrics demonstrating value. Once all pre-tasks are executed, then the next tasks should be recording the customer's decision and thanking them for their time and feedback, they should be done in parallel. Once all pre-tasks are executed, then you need to ask if they have any other concerns. When asking if they have any other concerns is completed, one or more of the following paths should be executed: sending learnings to appropriate managers, noting down important learnings from the conversation. The process is
now completed.

**DOT FORMAT**

```
digraph churn_rate_prevention_process_55 {
    graph [rankdir=LR]
    START_NODE [label="" shape=circle width=0.3]
    subgraph CLUSTER_0 {
        "record basic details" [shape=box]
        "OR_SPLIT--2. Evaluate product usage_IC9P" [label=O fixedsize=true shape=diamond width=0.5]
        "record basic details" -> "OR_SPLIT--2. Evaluate product usage_IC9P"
        "identify key points of value" [shape=box]
        "OR_SPLIT--2. Evaluate product usage_IC9P" -> "identify key points of value"
        "analyze how they use your product" [shape=box]
        "OR_SPLIT--2. Evaluate product usage_IC9P" -> "analyze how they use your product"
        "determine ways to increase value" [shape=box]
        "identify key points of value" -> "determine ways to increase value"
        "OR_JOIN--2. Evaluate product usage_IC9P" [label=O fixedsize=true shape=diamond width=0.5]
        "determine ways to increase value" -> "OR_JOIN--2. Evaluate product usage_IC9P"
        "analyze how they use your product" -> "OR_JOIN--2. Evaluate product usage_IC9P"
        "AND_SPLIT--3. Contact the customer_WKK6" [label="+" fixedsize=true shape=diamond width=0.5]
        "OR_JOIN--2. Evaluate product usage_IC9P" -> "AND_SPLIT--3. Contact the customer_WKK6"
        "record whether or not the customer responded" [shape=box]
        "AND_SPLIT--3. Contact the customer_WKK6" -> "record whether or not the customer responded"
        "send them an email" [shape=box]
        "AND_SPLIT--3. Contact the customer_WKK6" -> "send them an email"
        "confirm call date and time" [shape=box]
        "AND_SPLIT--3. Contact the customer_WKK6" -> "confirm call date and time"
        "AND_JOIN--3. Contact the customer_WKK6" [label="+" fixedsize=true shape=diamond width=0.5]
        "record whether or not the customer responded" -> "AND_JOIN--3. Contact the customer_WKK6"
        "send them an email" -> "AND_JOIN--3. Contact the customer_WKK6"
        "confirm call date and time" -> "AND_JOIN--3. Contact the customer_WKK6"
        "ask open-ended questions" [shape=box]
        "AND_JOIN--3. Contact the customer_WKK6" -> "ask open-ended questions"
        "AND_SPLIT--5. Retention incentives_22XV" [label="+" fixedsize=true shape=diamond width=0.5]
        "ask open-ended questions" -> "AND_SPLIT--5. Retention incentives_22XV"
        "present them with a special offer" [shape=box]
        "AND_SPLIT--5. Retention incentives_22XV" -> "present them with a special offer"
        "describe your ideas to increase value" [shape=box]
        "AND_SPLIT--5. Retention incentives_22XV" -> "describe your ideas to increase value"
        "inform them of key metrics demonstrating value" [shape=box]
        "AND_SPLIT--5. Retention incentives_22XV" -> "inform them of key metrics demonstrating value"
        "AND_JOIN--5. Retention incentives_22XV" [label="+" fixedsize=true shape=diamond width=0.5]
        "present them with a special offer" -> "AND_JOIN--5. Retention incentives_22XV"
        "describe your ideas to increase value" -> "AND_JOIN--5. Retention incentives_22XV"
        "inform them of key metrics demonstrating value" -> "AND_JOIN--5. Retention incentives_22XV"
        "AND_SPLIT--6. Outcome_PW4I" [label="+" fixedsize=true shape=diamond width=0.5]
        "AND_JOIN--5. Retention incentives_22XV" -> "AND_SPLIT--6. Outcome_PW4I"
        "record the customer's decision" [shape=box]
        "AND_SPLIT--6. Outcome_PW4I" -> "record the customer's decision"
        "thank them for their time and feedback" [shape=box]
        "AND_SPLIT--6. Outcome_PW4I" -> "thank them for their time and feedback"
        "AND_JOIN--6. Outcome_PW4I" [label="+" fixedsize=true shape=diamond width=0.5]
        "record the customer's decision" -> "AND_JOIN--6. Outcome_PW4I"
        "thank them for their time and feedback" -> "AND_JOIN--6. Outcome_PW4I"
        "ask if they have any other concerns" [shape=box]
        "AND_JOIN--6. Outcome_PW4I" -> "ask if they have any other concerns"
        "OR_SPLIT--8. Final steps_HCXR" [label=O fixedsize=true shape=diamond width=0.5]
        "ask if they have any other concerns" -> "OR_SPLIT--8. Final steps_HCXR"
        "send learnings to appropriate managers" [shape=box]
        "OR_SPLIT--8. Final steps_HCXR" -> "send learnings to appropriate managers"
        "note down important learnings from the conversation" [shape=box]
        "OR_SPLIT--8. Final steps_HCXR" -> "note down important learnings from the conversation"
        "OR_JOIN--8. Final steps_HCXR" [label=O fixedsize=true shape=diamond width=0.5]
        "send learnings to appropriate managers" -> "OR_JOIN--8. Final steps_HCXR"
        "note down important learnings from the conversation" -> "OR_JOIN--8. Final steps_HCXR"
    }
    START_NODE -> "record basic details"
    END_NODE [label="" shape=doublecircle width=0.2]
    "OR_JOIN--8. Final steps_HCXR" -> END_NODE}
```

(c) Complex Prompt

**INSTRUCTIONS**
Convert the following BPMN diagram in Graphviz DOT language into a valid BPMN XML file. The response must include only the XML, which should be parsable by its modeler tool and accurately represent the process. Follow these guidelines:

1. Include the graphical information for all elements (tasks, gateways, and edges), including coordinates and sequence flow connections. Ensure waypoints are included to prevent overlapping flows or elements.
2. Arrange elements:
   - Sequential tasks should align horizontally from left to right.
   - Parallel and alternative paths should be arranged vertically for clarity.
3. Accurately represent gateway types:
   - Exclusive Gateway (XOR): Only one path can be taken (e.g., "checking for completeness").
   - Inclusive Gateway (OR): One or more paths can be taken (e.g., "one or more of the following paths should be executed").
   - Parallel Gateway (AND): All paths must be taken simultaneously.
4. Add corresponding merging gateways for every split gateway to complete the structure before merging paths.
5. Properly connect all elements, ensuring sequence flow accurately reflects the graph and process description.
6. Maintain adequate spacing between elements and gateways to improve readability, avoiding overlapping arrows.
7. Represent each task, gateway, and edge as defined in the provided process description and Graphviz DOT language code.
Below is the process description and DOT representation of the graph. Use both as-is (do not rephrase or modify the content) to generate the XML

**PROCESS DESCRIPTION**
The following text is about the customer support process for the ticket management. It starts with making an introduction. When making an introduction is completed, one or more of the following paths should be executed: creating accounts, getting approval, assigning required reading materials. After getting approval, the process is split into 3 parallel branches: preparing employee paperwork, giving access to any tools they will need and preparing benefits package. Once assigning required reading materials occurs, the next tasks should be providing a job description with responsibilities, discussing role, goals and projects with supervisor and preparing employee's workstation, they should be done in parallel. After that, one or more of the following paths should be executed: taking out to lunch, explaining your expectations. Once explaining your expectations occurs, holding a welcoming ceremony should be done. After that, you need to get approval. Once getting approval occurs, providing reading material for personal growth should be done. The process is now completed.

**DOT FORMAT**

```
digraph customer_support_process_for_the_ticket_management_194 {
    graph [rankdir=LR]
    START_NODE [label="" shape=circle width=0.3]
    subgraph CLUSTER_0 {
        "make an introduction" [shape=box]
        "OR_SPLIT--2. Before first day_SY7O" [label=O fixedsize=true shape=diamond width=0.5]
        "make an introduction" -> "OR_SPLIT--2. Before first day_SY7O"
        "create accounts" [shape=box]
        "OR_SPLIT--2. Before first day_SY7O" -> "create accounts"
        "get approval" [shape=box]
        "OR_SPLIT--2. Before first day_SY7O" -> "get approval"
        "assign required reading materials" [shape=box]
        "OR_SPLIT--2. Before first day_SY7O" -> "assign required reading materials"
        "AND_SPLIT--2. Before first day_66JB" [label="+" fixedsize=true shape=diamond width=0.5]
        "get approval" -> "AND_SPLIT--2. Before first day_66JB"
        "prepare employee paperwork" [shape=box]
        "AND_SPLIT--2. Before first day_66JB" -> "prepare employee paperwork"
        "give access to any tools they will need" [shape=box]
        "AND_SPLIT--2. Before first day_66JB" -> "give access to any tools they will need"
        "prepare benefits package" [shape=box]
        "AND_SPLIT--2. Before first day_66JB" -> "prepare benefits package"
        "AND_JOIN--2. Before first day_66JB" [label="+" fixedsize=true shape=diamond width=0.5]
        "prepare employee paperwork" -> "AND_JOIN--2. Before first day_66JB"
        "give access to any tools they will need" -> "AND_JOIN--2. Before first day_66JB"
        "prepare benefits package" -> "AND_JOIN--2. Before first day_66JB"
        "AND_SPLIT--2. Before first day_O1AK" [label="+" fixedsize=true shape=diamond width=0.5]
        "assign required reading materials" -> "AND_SPLIT--2. Before first day_O1AK"
        "provide a job description with responsibilities" [shape=box]
        "AND_SPLIT--2. Before first day_O1AK" -> "provide a job description with responsibilities"
        "discuss role, goals and projects with supervisor" [shape=box]
        "AND_SPLIT--2. Before first day_O1AK" -> "discuss role, goals and projects with supervisor"
        "prepare employee's workstation" [shape=box]
        "AND_SPLIT--2. Before first day_O1AK" -> "prepare employee's workstation"
        "AND_JOIN--2. Before first day_O1AK" [label="+" fixedsize=true shape=diamond width=0.5]
        "provide a job description with responsibilities" -> "AND_JOIN--2. Before first day_O1AK"
        "discuss role, goals and projects with supervisor" -> "AND_JOIN--2. Before first day_O1AK"
        "prepare employee's workstation" -> "AND_JOIN--2. Before first day_O1AK"
        "OR_JOIN--2. Before first day_SY7O" [label=O fixedsize=true shape=diamond width=0.5]
        "create accounts" -> "OR_JOIN--2. Before first day_SY7O"
        "AND_JOIN--2. Before first day_66JB" -> "OR_JOIN--2. Before first day_SY7O"
        "AND_JOIN--2. Before first day_O1AK" -> "OR_JOIN--2. Before first day_SY7O"
        "OR_SPLIT--3. First day_2FOU" [label=O fixedsize=true shape=diamond width=0.5]
        "OR_JOIN--2. Before first day_SY7O" -> "OR_SPLIT--3. First day_2FOU"
        "take out to lunch" [shape=box]
        "OR_SPLIT--3. First day_2FOU" -> "take out to lunch"
        "explain your expectations" [shape=box]
        "OR_SPLIT--3. First day_2FOU" -> "explain your expectations"
        "hold a welcoming ceremony" [shape=box]
        "explain your expectations" -> "hold a welcoming ceremony"
        "OR_JOIN--3. First day_2FOU" [label=O fixedsize=true shape=diamond width=0.5]
        "take out to lunch" -> "OR_JOIN--3. First day_2FOU"
        "hold a welcoming ceremony" -> "OR_JOIN--3. First day_2FOU"
        "get approval" [shape=box]
        "OR_JOIN--3. First day_2FOU" -> "get approval"
        "provide reading material for personal growth" [shape=box]
        "get approval" -> "provide reading material for personal growth"
    }
    START_NODE -> "make an introduction"
    END_NODE [label="" shape=doublecircle width=0.2]
    "provide reading material for personal growth" -> END_NODE}
```

**Author contributions**
All authors contributed to the study conception and design. All authors have participated in material preparation, data collection and analysis. All authors read and approved the final manuscript.

**Funding**
This research received no external funding.

**Data availability**
The datasets used are available from the corresponding author on reasonable request.

## Declarations

**Competing interests**
The authors declare that they have no competing interests.

Received: 30 August 2025 / Accepted: 3 November 2025

Published online: 20 November 2025

**References**


1. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901
3. Busch K. & Leopold H. (2024) Towards a Benchmark for Large Language Models for Business Process Management Tasks, arXiv, vol. 2410,
2. Busch K., Rochlitzer A., Sola D., & Leopold H. Just tell me: Prompt engineering in business process management. arXiv. 2023. https://arxiv.org/abs/2304.07183
4. Chen L. & Varoquaux G. (2024) What is the role of small models in the LLM era: a survey. arXiv
5. Friedrich F., Mendling J., & Puhlmann F. (2011) Process model generation from naturallanguage text, In: Advanced Information Systems Engineering – 23rd InternationalConference, CAiSE. Lecture Notes in Computer Science, vol. 6741, pp. 482-4
6. Fu, X.-Y., Laskar, M. T. R., Khasanova, E., Chen, C., & Bhushan, S. T. N. (2024) Tiny titans: can smaller large language models punch above their weight in the real world for meeting summarization? arXiv
7. Gao, T., Fisch, A., & Chen, D. (2021) Making pre-trained Language models better few-shot learners. Proc 59th Annual Meeting Association Comput Linguistics (ACL) 2021; 3816–3830. https://aclanthology.org/2021.acl-long.295
8. Grohs, M., Abb, L., Elsayed, N., & Rehse, J.-R. (2023) Large language models can accomplish business process management tasks. arXiv
9. Han, X., Hu, L., Dang, Y., Agarwal, S., Mei, L., Li, S., & Zhou, X. (2020) Automatic business process structure discovery using ordered neurons LSTM: a preliminary study. arXiv
10. Hillier, D., Guertler, L., Tan, C., Agrawal, P., Ruirui, C., & Cheng, B. (2024) Super tiny language models. arXiv. https://arxiv.org/abs/2405.14159
11. Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J., & Lin, J. (2024) Qwen2.5-coder technical report. arXiv
12. Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022) Large language models are zero-shot reasoners. Adv Neural Inf Process Syst 35:22199–22213
13. Kourani, H., Berti, A., Schuster, D *et al* (2025) Evaluating large Language models on business process modeling: framework, benchmark, and self-improvement analysis. Softw Syst Model. https://doi.org/10.1007/s10270-025-01318-w
15. Li, X., Ni, L., Li, R., Liu, J., & Zhang, M. (2023) MaD: A dataset for interview-based BPM in business process management. In: 2023 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. https://doi.org/10.1109/IJCNN54540.2023.10191898
16. Licardo, J. T., Tankovic, N., & Etinger, D. (2025) BPMN assistant: an LLM-based approach to business process modeling. arXiv
14. Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023) Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. ACM-CSUR 55(9):1–35. https://doi.org/10.1145/3560815
17. Lu, Z., Li, X., Cai, D., Yi, R., Liu, F., Zhang, X., Lane, N. D., & Xu, M. (2025) Small language models: survey, measurements, and insights. arXiv
18. Nour Eldin, A., Assy, N., Anesini, O., Dalmas, B., & Gaaloul, W. (2025) Nala2BPMN: Automating BPMN model generation with large Language models. In: Comuzzi M, Grigori D, Sellami M, Zhou Z (eds) Cooperative information Systems. CoopIS 2024. Lecture Notes in Computer Science, vol 15506. Springer, Cham. https://doi.org/10.1007/978-3-031-81375-7_27.
19. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019) Language models are unsupervised multitask learners. OpenAI Technical Report. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
20. Reynolds, L. & McDonell, K. (2021) Prompt programming for large Language models: beyond the few-shot paradigm. ArXiv Preprint arXiv:2102.07350. https://arxiv.org/abs/2102.07350
21. Scao, T. L. & Rush, A. M. (2021) How many data points is a prompt worth? Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2627–2636. https://aclanthology.org/2021.naacl-main.208
22. Schick, T. & Schütze, H. (2021) It's not just size that matters: Small language models are also few-shot learners. Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2339–2352. https://aclanthology.org/2021.naacl-main.185
23. Sholiq, S., Sarno, R., & Astuti, E. S. (2022) Generating BPMN diagram from textual requirements. Journal of King Saud University - Computer and Information Sciences, 34(10, Part B), 10079–10093. https://doi.org/10.1016/j.jksuci.2022.10.007
24. Unsloth (2025) Fine-tuning guide. https://docs.unsloth.ai/get-started/fine-tuning-guide. Accessed 12 July 2025.
25. Unsloth AI, Han-Chen, D., & Han-Chen, M. (2025) Qwen2.5-14B-Instruct [Model]. Hugging Face. https://huggingface.co/unsloth/Qwen2.5-14B-Instruct Accessed 12 July 2025
26. Van Nguyen, C., Shen, X., Aponte, R., Xia, Y., Basu, S., Hu, Z., Chen, J., Parmar, M., Kunapuli, S., Barrow, J., Wu, J., Singh, A., Wang, Y., Gu, J., Dernoncourt, F., Ahmed, N. K., Lipka, N., Zhang, R., Chen, X., Yu, T., Kim, S., Deilamsalehy, H., Park, N., Rimer, M., Zhang, Z., Yang, H., Rossi, R. A., & Nguyen, T. H. (2024) A survey of small language models. arXiv. https://arxiv.org/abs/2410.20011
27. Vidgof, M., Bachhofner, S., & Mendling, J. (2023) Large language models for business process management: Opportunities and challenges. arXiv. https://arxiv.org/abs/2304.04309
28. Wang, F., Zhang, Z., Zhang, X., Wu, Z., Mo, T., Lu, Q., Wang, W., Li, R., Xu, J., Tang, X., He, Q., Ma, Y., Huang, M., & Wang, S. (2024) *A* comprehensive survey of small language models in the era of large language models: techniques, enhancements, applications, collaboration with LLMs, and trustworthiness. arXiv
29. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., & Zhou, D. (2022) Chain-of-thought prompting elicits reasoning in large Language models. Adv Neural Inf Process Syst 35:24824–24837
30. Xu, F., Uszkoreit, H., Du, Y., Fan, W., Zhao, D., & Zhu, J. (2019) Explainable AI: A brief survey on History, research Areas, approaches and challenges. In: Tang J, Kan MY, Zhao D, Li S, Zan H (eds) Natural Language processing and Chinese

Computing. NLPCC 2019. Lecture notes in computer Science, vol 11839. Springer, Cham. https://doi.org/10.1007/978-3-030-32236-6_51.
31. Xu, L., Xie, H., Qin, S., Tao, X., & Wang, F. L. (2023) Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment. arXiv preprint arXiv:2312.12148
32. Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., & Chi, E. H. (2022) Least-to-most prompting enables complex reasoning in large language models. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2205.10625

## Publisher's note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.